FAKE JOB POSTING DETECTION

Detecting Fraudulent Job Postings

Leveraging NLP for Enhanced Security

Presented to:

Mr. Praveen

Presented By:

Palkin

OVERVIEW

The Growing Threat of Recruitment Fraud

> Current fraud detection models are often static, lacking real-world security, live monitoring, and the ability to learn from evolving scam patterns.

THE PROBLEM

Key Challenges in Fraud Detection

Recruitment Fraud Surge

Escalation of sophisticated fake job postings designed to harvest personal data or financial assets.

Deployment Gap

Existing ML models often remain as local scripts, lacking the infrastructure for real-world deployment and scalability.

Security Vulnerabilities

Absence of secure administrative access control in many standard detection tools, increasing risk.

Model Stagnation

Static models fail to adapt quickly to evolving scam tactics and new fraud patterns after initial training.

PROJECT FOCUS

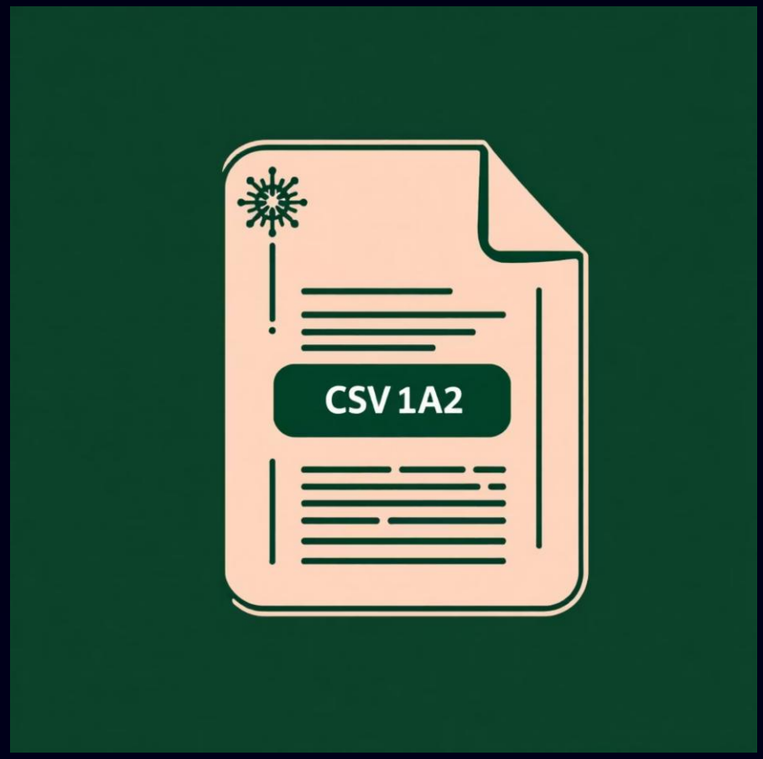
Objective & Data Source

Primary Objective

To develop a robust system for detecting fraudulent job postings utilizing advanced Natural Language Processing (NLP) techniques.

Dataset Details

- **Source:** Kaggle
- **File:** `fake_job_postings.csv`
- **Rows:** 17,880
- **Columns:** 18 diverse features
- **Target Variable:** 'fraudulent' (binary classification)



MILESTONE 1

Data Intelligence & Exploratory Analysis

Data Preprocessing & Cleaning

- Removal of HTML tags, special characters, and text normalization (lowercasing, tokenization).
- Efficient handling of missing values and optimization of data types.

Exploratory Data Analysis (EDA)

- Statistical analysis and visualization using Word Clouds and Distribution Plots.
- Identified significant class imbalance between real and fake listings.

Feature Engineering Pipeline

- Implemented TF-IDF Vectorization for converting text into numerical features.
- Established a reproducible pipeline for consistent model input and scaling.

MILESTONE 1: DETAILS

Comprehensive Data Cleaning Steps

Our initial phase involved rigorous data cleaning to ensure the highest quality input for our NLP models. This foundational step is crucial for accurate fraud detection.

HTML Tag Removal

Eliminated all HTML elements from job descriptions.

URL & Special Character Scrubbing

Removed all embedded URLs and irrelevant special characters.

Lowercasing & Whitespace Normalization

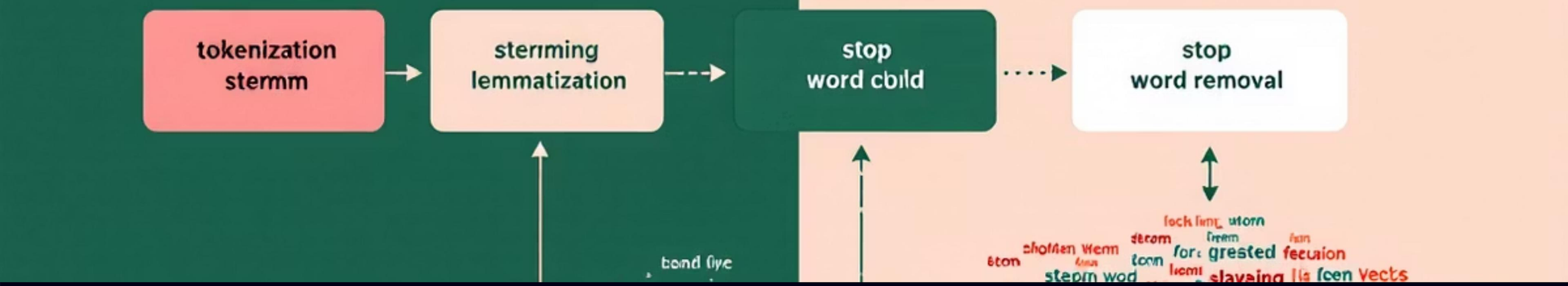
Standardized text to lowercase and cleaned up excessive whitespace.

Missing Value Imputation

Strategically handled and filled in missing data points.

Text Field Merging

Combined relevant text fields into a single, comprehensive column for unified analysis.



MILESTONE 1: DETAILS

Advanced Text Normalization Techniques

Tokenization

Breaking down text into individual words or sub-word units.

Stopword Removal

Eliminating common words (e.g., "the," "is") that add noise to analysis.

Lemmatization

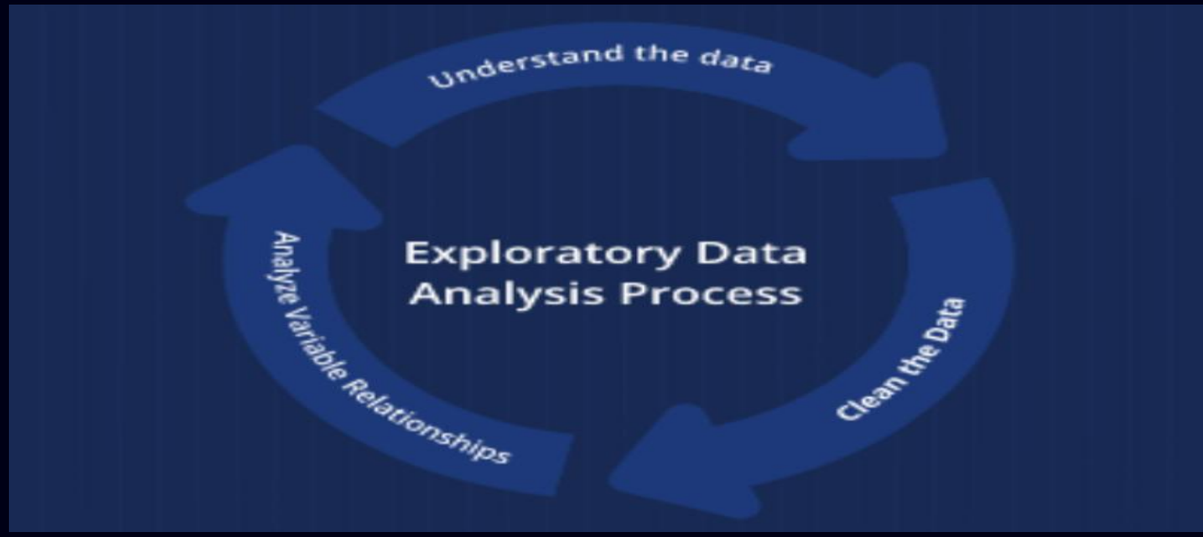
Reducing words to their base or dictionary form (e.g., "running" to "run").

Custom Preprocessing Pipeline

Developed a tailored pipeline for consistent and efficient text preparation.

MILESTONE 1: DETAILS

Key Statistical Findings from EDA



5%

Fake Job Posts

Only a small percentage of total listings were fraudulent, highlighting the class imbalance issue.

High

Missing Values

Salary range and benefits fields showed the highest rates of missing information.

Sparse

Categorical Fields

Many categorical attributes contained sparse data, complicating direct analysis.

These statistical insights underscore the necessity for advanced techniques to address data quality and distribution challenges, particularly the severe class imbalance.

MILESTONE 1: DETAILS

Visual Highlights from EDA

- Authenticity in Language

  Real job postings frequently use specific, technical terminology.

- Fraudulent Language Patterns

  Fake postings often feature vague, promotional, or overly generic terms.

- Geographical Bias

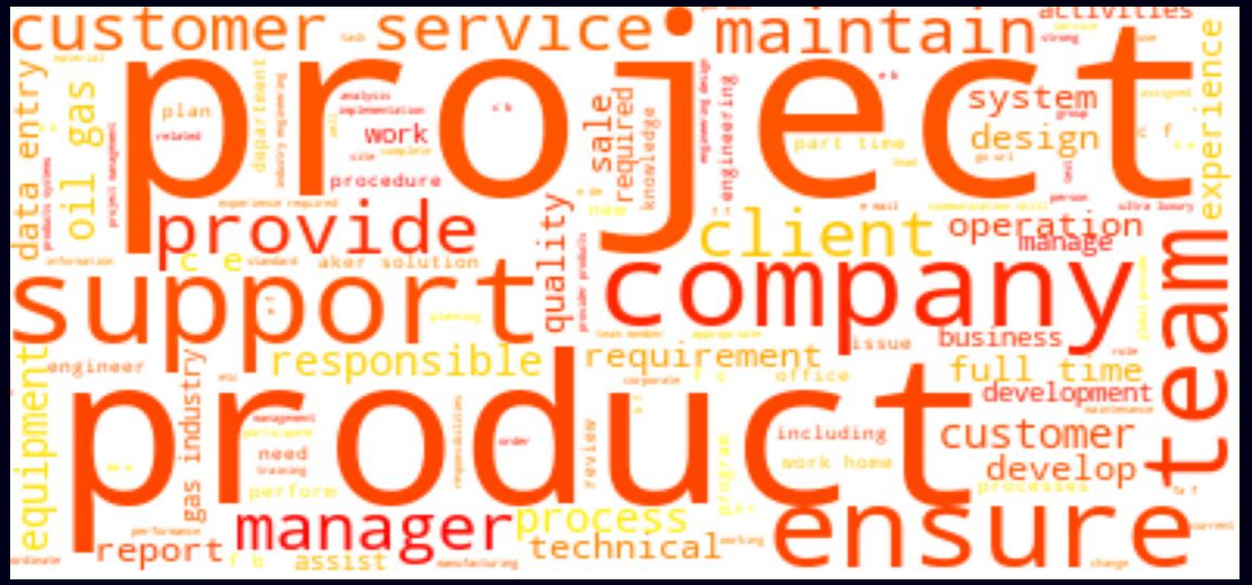
  Location distribution is heavily concentrated in US and UK regions.

- Distinct Word Clouds

  Visual word clouds distinctly highlight linguistic differences between legitimate and fraudulent posts.

Real Job Postings

Fake Job Postings



CRITICAL FINDING

Addressing Class Imbalance

Identified Distribution

- **Real Job Postings:** Approximately 95%
- **Fake Job Postings:** Approximately 5%

The Challenge: This severe imbalance can lead to models that are biased towards the majority class, poorly detecting actual fraudulent cases.

Strategies for Milestone 2

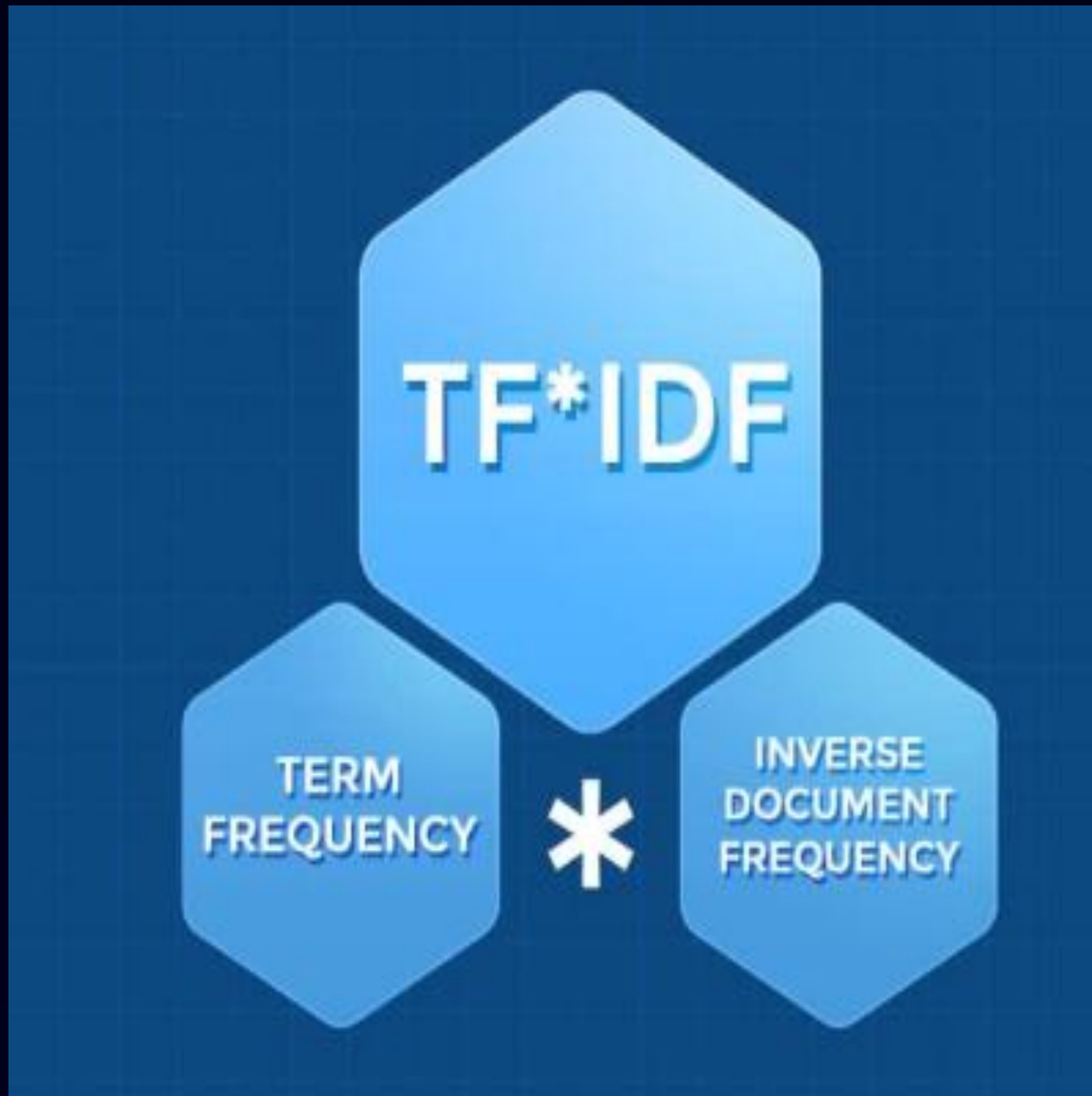
- **SMOTE (Synthetic Minority Over-sampling Technique):** To generate synthetic samples for the minority class.
- **Oversampling:** Duplicating existing minority class samples.
- **Class-weighting:** Adjusting the importance of classes during model training.

TF-IDF Feature Extraction

What TF-IDF Does:
- Converts text into numerical weighted features.
- Measures importance of words based on frequency.

How We Applied It:
- Used sklearn TfidfVectorizer
- max_features = 5000 for efficiency
- ngram_range = (1,2) to capture bigrams
- min_df = 5 to ignore rare noise words
- stop_words='english'

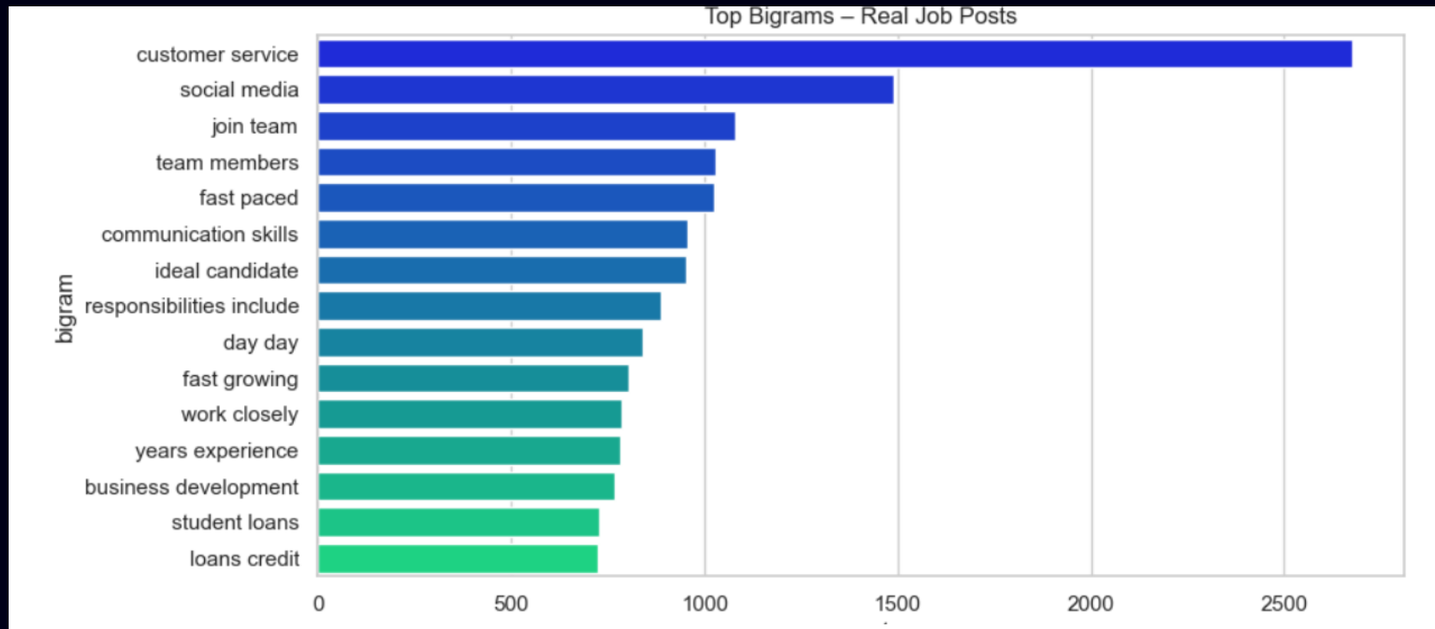
Outputs:
- Sparse matrix of shape (17880, 5000)
- Ready for ML models in Milestone 2

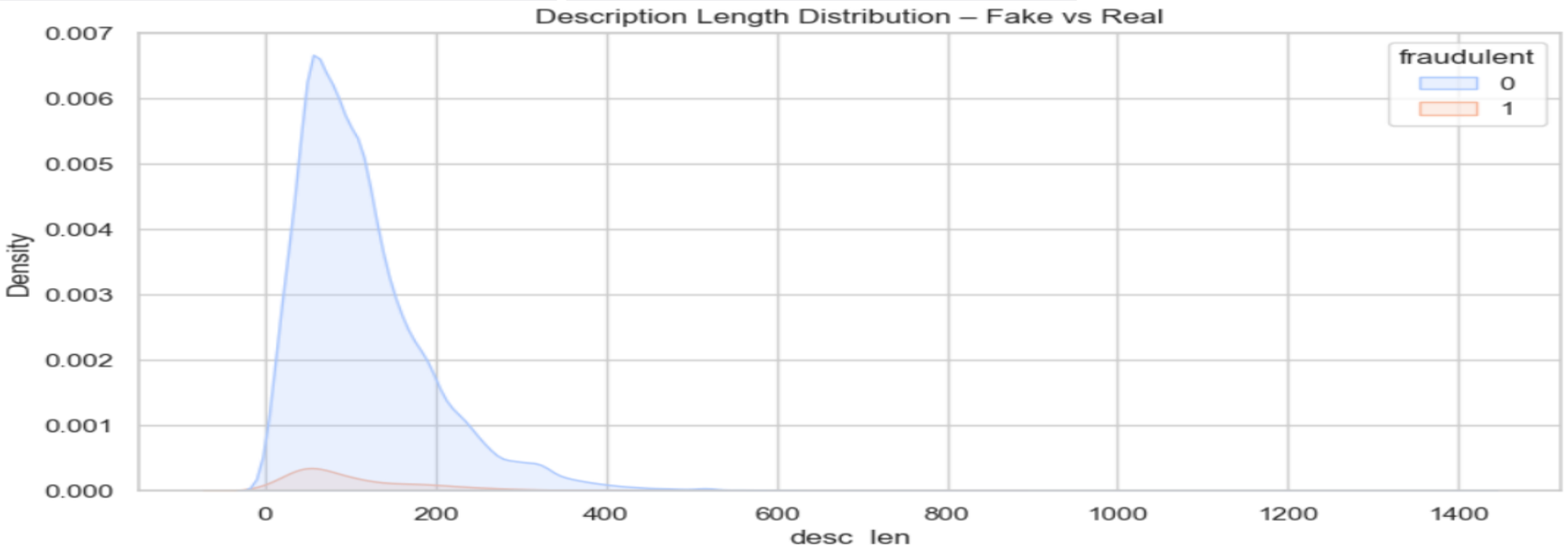


Milestone 1 Outputs

Deliverables Completed:

- ✓ Cleaned dataset
- ✓ Normalized text
- ✓ EDA visuals
- ✓ TF-IDF vector matrix
- ✓ Preprocessing pipeline
- ✓ Notebook ready for submission

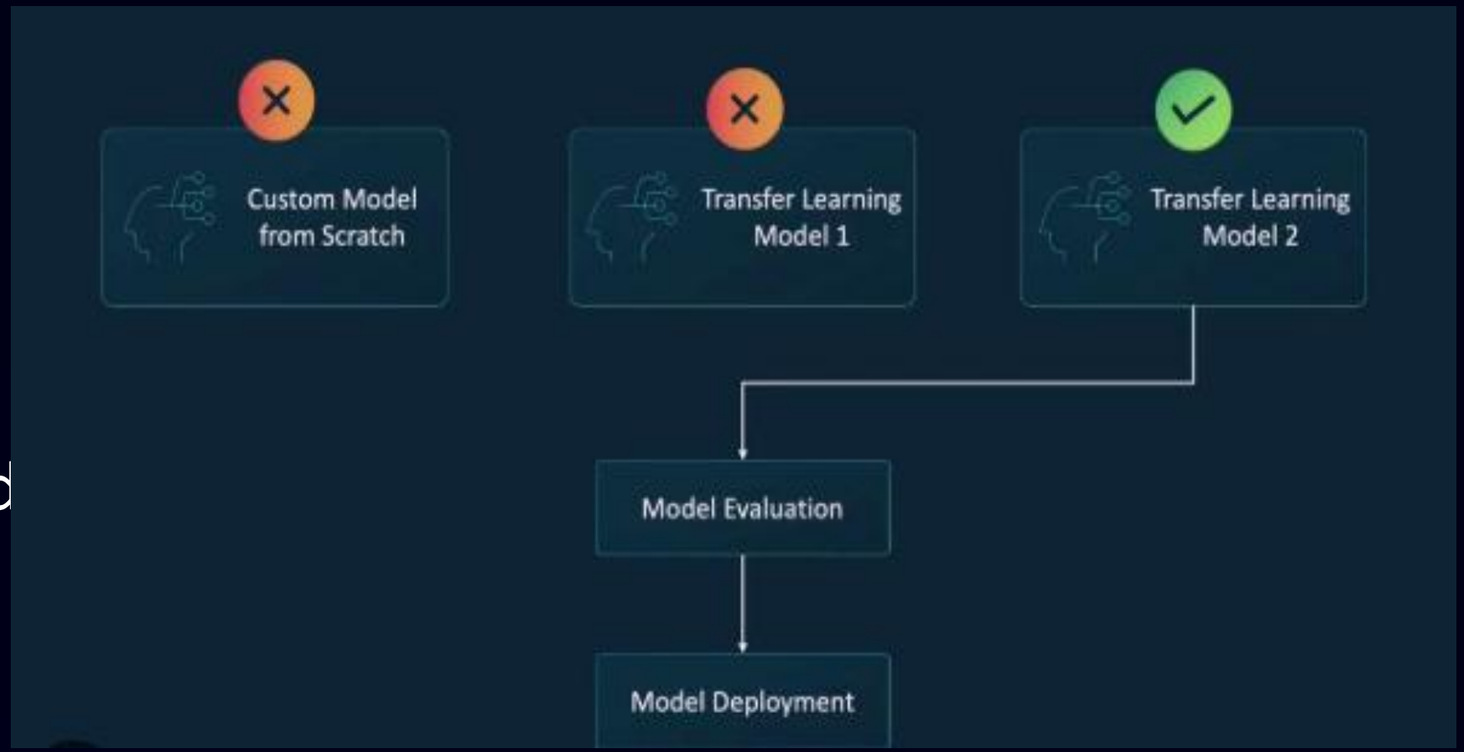




Milestone 2 Overview

Model Training & Evaluation

- Baseline ML models implemented
- Deep learning architecture developed
- Models compared and evaluated
- Best model selected & saved
- Complete performance analysis documented



Baseline Models Implemented

Logistic Regression
- Trained on TF-IDF features
- Hyperparameters tuned (C, penalty, solver)
- Fast training time recorded
- Saved as .pkl

Random Forest
- Trained with optimized n_estimators & max_depth
- Feature importance extracted
- Saved for final comparison

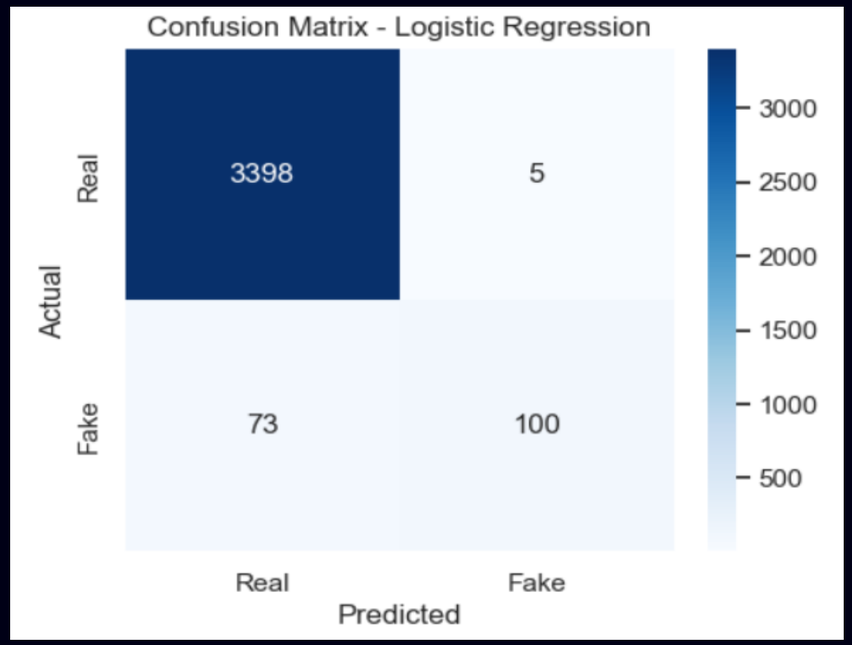
Evaluation of Baseline Models

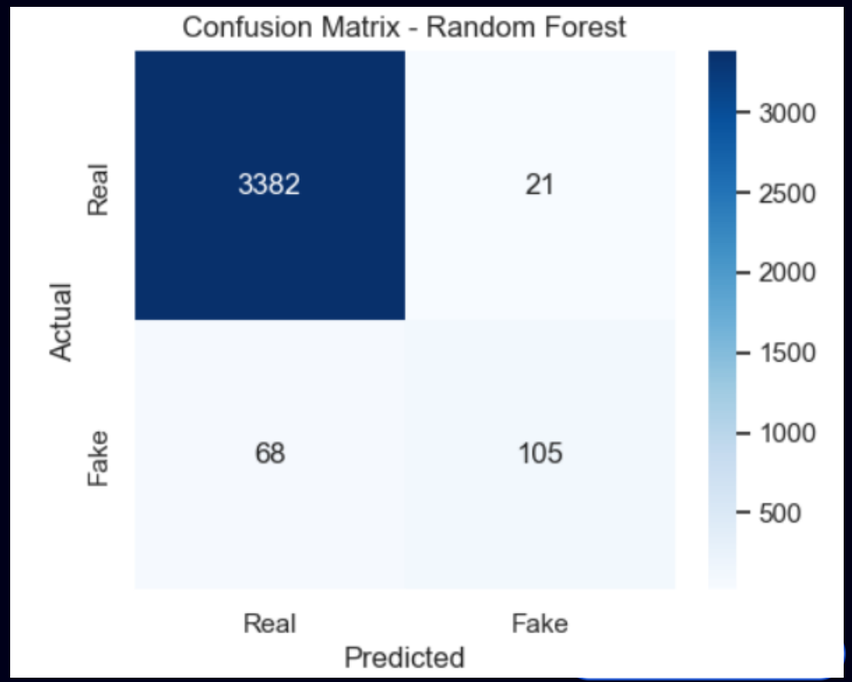
Metrics Calculated for Both Models

- Accuracy score
- Precision, Recall, F1-score
- Confusion matrix
- ROC Curve + AUC
- Classification report summarized

Observation:

Models provide strong baseline performance for next stage.





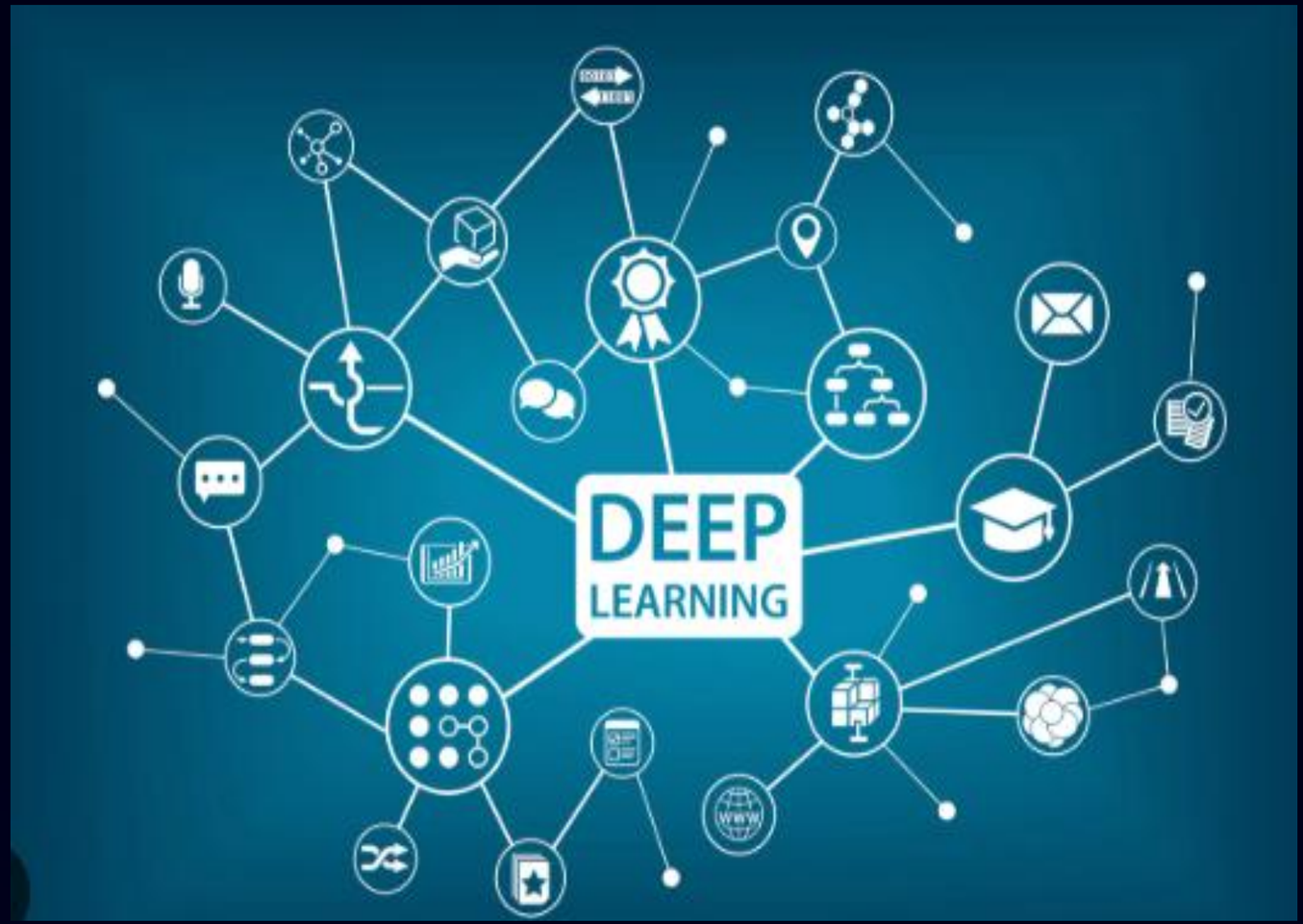
Deep Learning Model: Architecture

Chosen Architecture: BiLSTM

- Input sequences pre-padded
- Embedding layer (Keras / GloVe embeddings)
- BiLSTM layer for contextual understanding
- Dense + Dropout layers for regularization
- Sigmoid output for binary classification

Why BiLSTM:

- Captures long-range patterns
- Performs better on textual data
- Balanced complexity vs performance



Deep Learning: Training Process

- Compiled using binary_crossentropy
- Optimizer: Adam
- Train/validation split maintained
- Epochs & batch size optimized
- Early stopping applied
- Training logs saved (loss & accuracy curves)

Deep Learning: Evaluation

Performance Achieved

Accuracy > **90%**

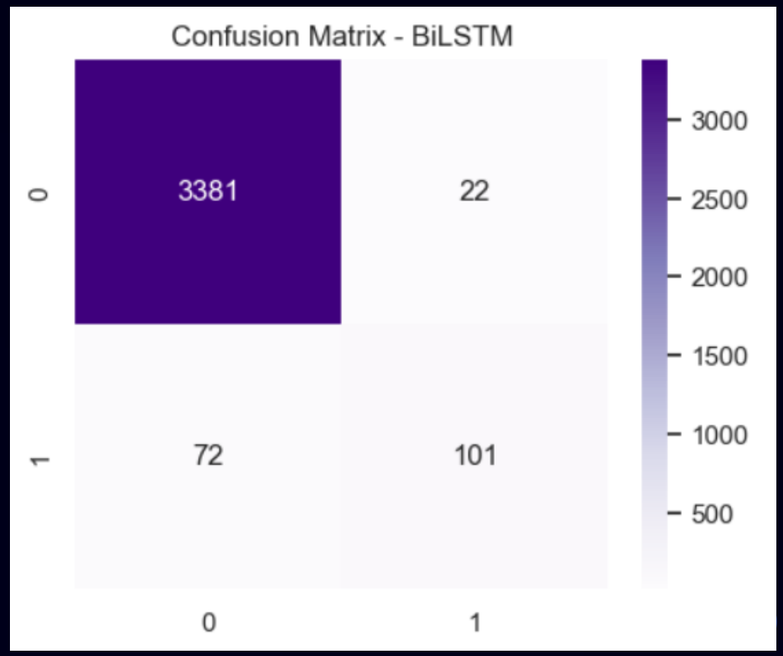
High and balanced F1-score

No major overfitting observed

Clear convergence in training curves

Conclusion:

Deep learning outperformed baseline models.



Model Comparison

Model	Accuracy	F1-Score	Training Time	Complexity
Logistic Regression	Good	Good	Very Fast	Low
Random Forest	Very Good	Very Good	Fast	Medium
BiLSTM	Best	Best	High	High

Selected Best Model: BiLSTM

- Highest accuracy and F1-score
- Most stable performance
- Best for real-world deployment

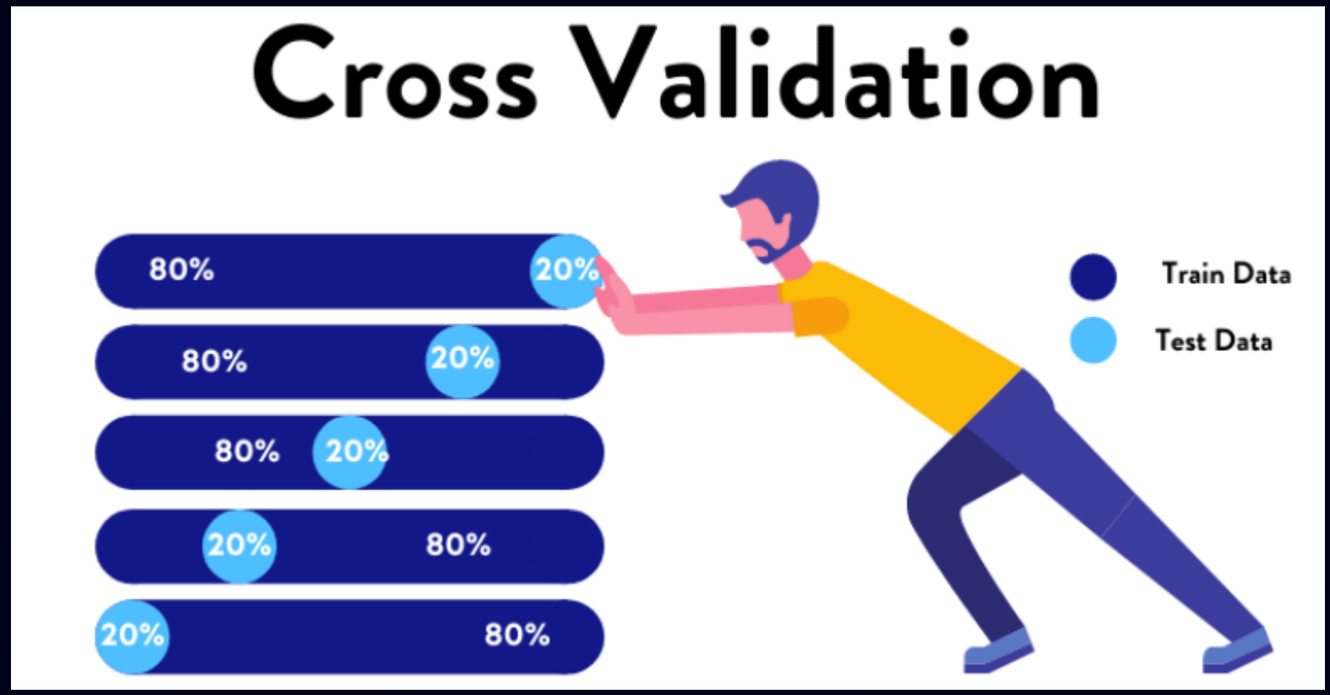
MODEL SAVING & VERSIONING

- Best Model Saved As **Model_v1.0.H5**
- Metadata Stored (Accuracy, Hyperparameters, Embedding Info)
- Preprocessing & Inference Pipeline Validated
- Version Control Maintained For Reproducibility

CROSS-VALIDATION

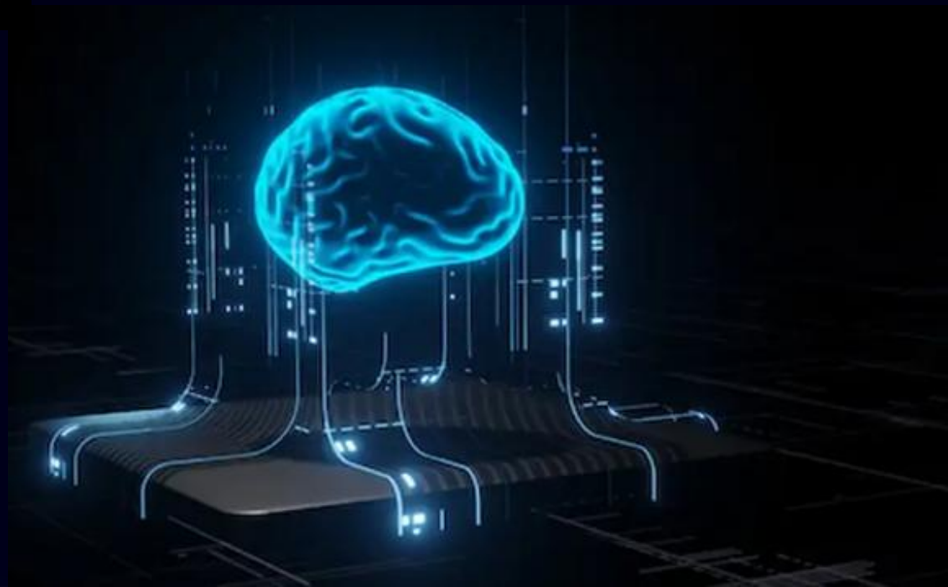
- K-Fold Cross-Validation (k = 5/10) performed
- Low variance across folds
- Stable model performance observed
- Confirms robustness of selected model

Model Saving & Versioning



MILESTONE 2 SUMMARY

- Baseline models built & evaluated
- Deep learning model developed (Bi LSTM)
- Achieved >90% accuracy
- Complete comparison and selection done
- Best model saved with versioning
- Ready for Milestone 3 (Deployment & API)

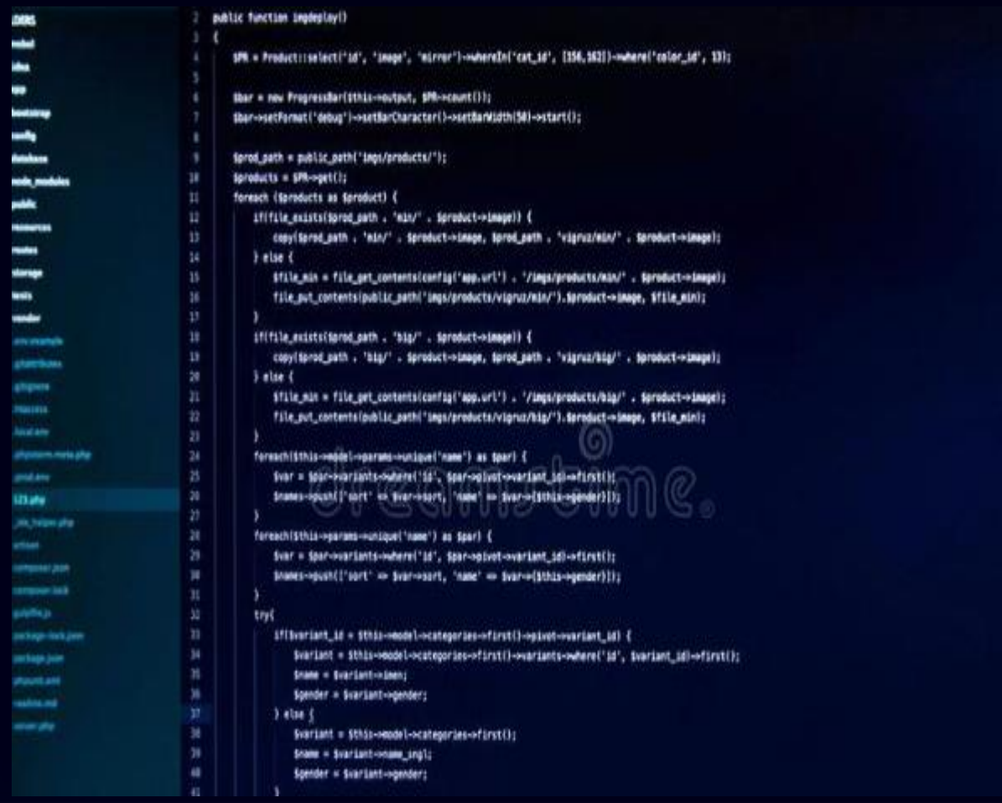


Milestone 3: Operationalizing the Model – Web Application Integration

- **Backend API Development (M3):** Develop a robust, secure
- inference API using Fast API.

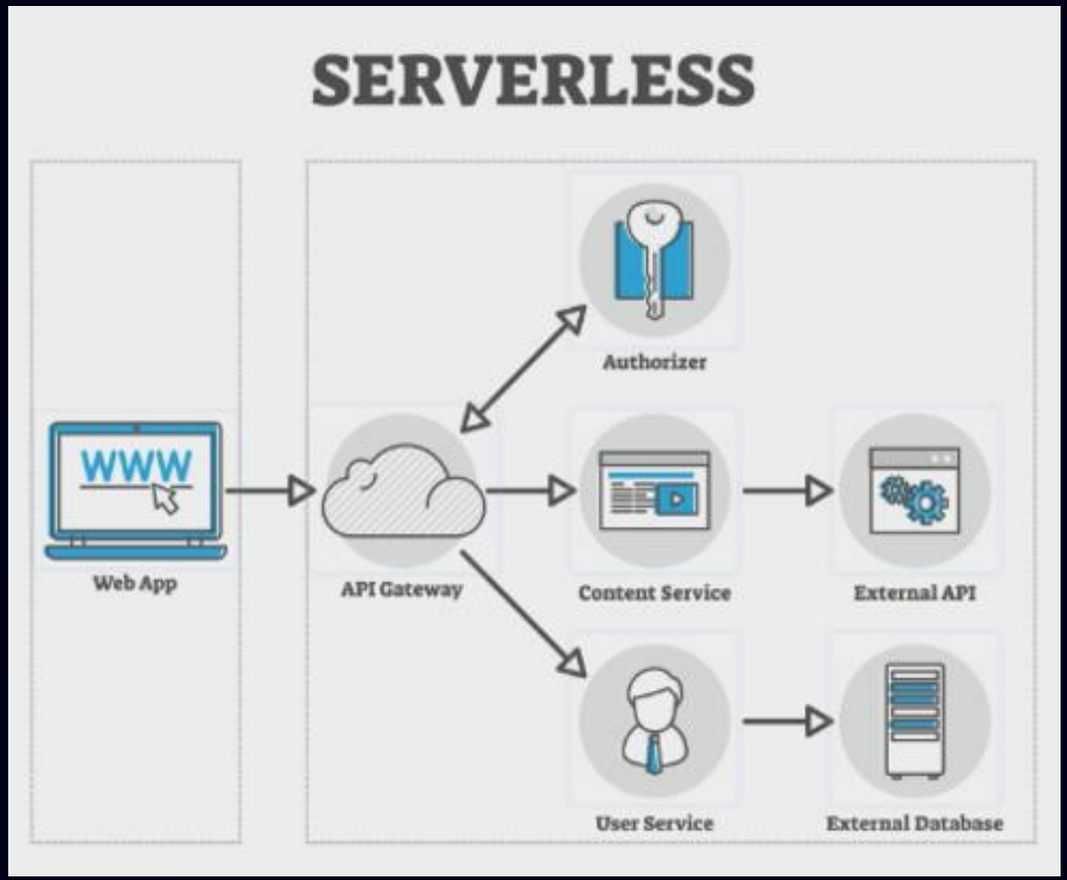
- **Frontend Interface (M3):** Create an intuitive, responsive UI for
- user interaction and clear result display.

- **Feedback & Logging (M3):** Implement a continuous feedback
- mechanism and secure data logging for future model maintenance.



Production Architecture: The End-to-End System Flow

Seamlessly connecting the high-performance ML model selected in Milestone 2 with a scalable web framework (FastAPI).



ROBUST BACKEND: LEVERAGING FAST API FOR HIGH PERFORMANCE

- **Selected Fast API for its asynchronous capabilities (high concurrency) and automatic Swagger/Open API documentation.**

Core Logic (main.py):

•**Model Initialization:** Models are loaded globally outside of any route, ensuring they are loaded only once on server startup, maximizing response speed.

•**Dependency Injection (DI):** Utilized for managing security and user roles (Depends(get _ current _ user)), making the code modular and testable.

•**CORS Management:** Configured CORS Middleware to allow cross-origin requests, enabling the local index.html to communicate with the Fast API server.

Model Inference Endpoint and Data Logging

- **Endpoint:** /predict (POST)
- **Input Handling:** Accepts JSON body adhering to the Job Input Pydantic model (description: str).
- **Input Validation:** Raises HTTP Exception(400) if the input text is too short (<10 characters).
- **Prediction Logic:**

1. Input text vectorized: vectorizer.transform([data.description]).
2. Prediction made: model.predict(vec)[0].
3. Confidence derived: max(model.predict_proba(vec)[0]).

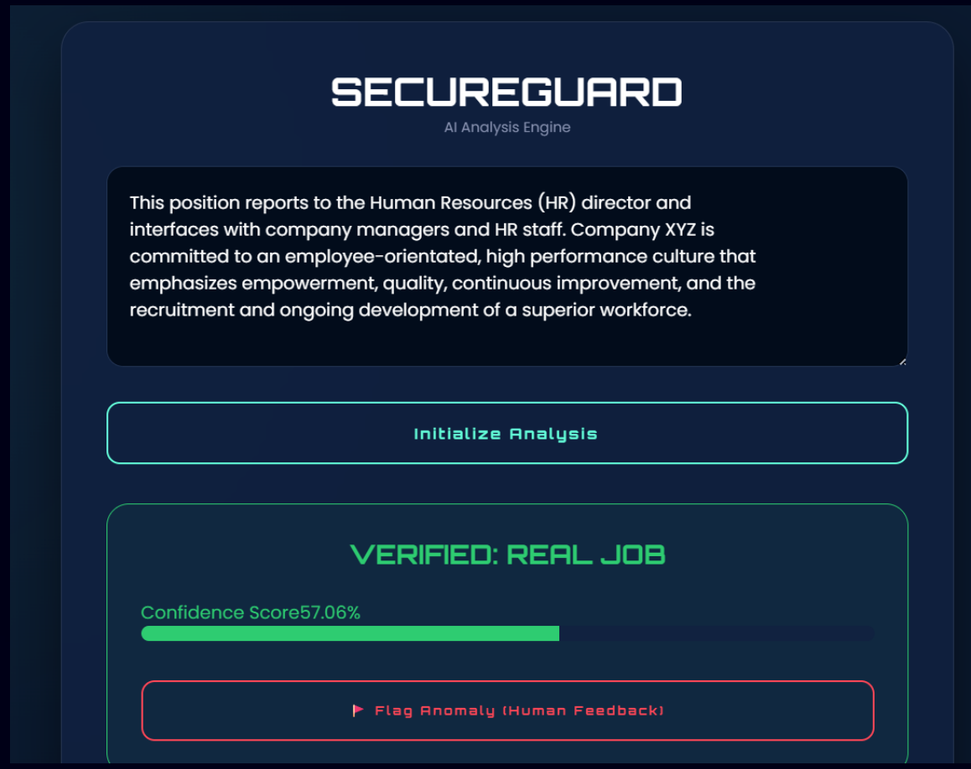
- **Logging Activity:** Every request is logged to **prediction_logs.csv**
- with timestamp, description_length, prediction, and confidence—providing raw data for dashboard statistics.

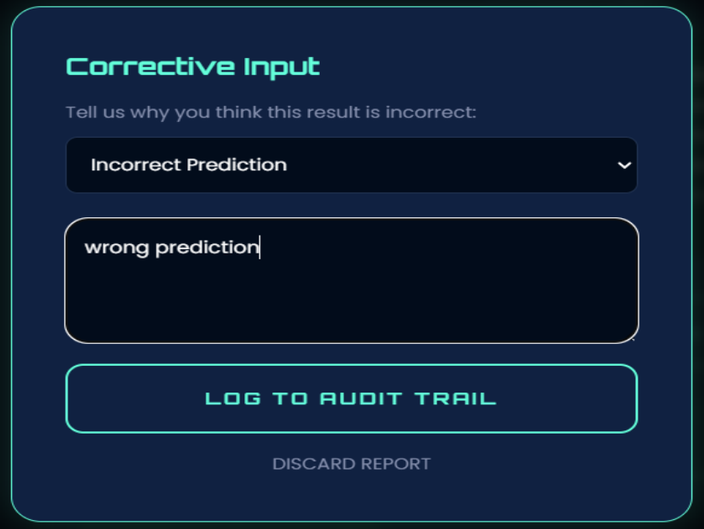
Intuitive Frontend: Design and Dynamic Results

- **Design Focus:** Clean, single-page application with immediate, color-coded feedback.
- **Dynamic Styling:**
- If **Result: 'Fake'**: result-box class changes to Fake (Red/Pink gradient) with icon ⚠.
- If **Result: 'Real'**: result-box class changes to Real (Green gradient) with icon ✓.
- **JavaScript Handling:**
- Asynchronous predict() function manages UI state transitions (input → loading spinner → result display).
- Uses fetch API to communicate with http://127.0.0.1:8000/predict.
- **Clear Display:** Confidence Score is clearly displayed alongside the classification result.

THE FEEDBACK MECHANISM AND DATA COLLECTION

- **Goal:** Establish a critical data collection point to mitigate model drift and improve future retraining batches.
- **Mechanism:**
- **Frontend:** A "⚑ Report Incorrect Prediction" button triggers a form with a dropdown for reason and text input for comments.
- **Endpoint:** /feedback (POST) accepts Feedback Input model data.
- **Persistence:** Data is appended to **flagged_jobs.csv**.
- **Stored Data Fields:** timestamp, reason, predicted (what the model predicted), snippet (first 50 chars of description), and comments. This data is the **human-labeled ground truth** needed for the next model iteration.





SECURE SYSTEM MONITORING AND ANALYTICS

•**Goal:** Provide authenticated administrators with key insights and system control.

•**Authentication:** Utilizes Fast API's secure

•**OAuth2PasswordBearer** scheme and JWT for token creation and verification (get _current _ user). Access is restricted to users with role: "admin".

•**Statistics Endpoint (/admin/stats):**
- •Pulls and aggregates data from prediction_logs.csv.
- •Returns metrics: total, fake count, real count, and the current **model_version**.
- •Provides real-time visibility into system usage and class distribution.

Model Maintenance Controls

- **Goal:** Enable administrators to manage the model's lifecycle.
- **Flagged Jobs Review (/admin/flagged-jobs):**
- Returns the contents of **flagged_jobs.csv** (the human feedback data) as a list of dictionaries for review.
- **Data Export (/admin/export):**
- Serves the flagged_jobs.csv file directly as a downloadable **FileResponse**. This facilitates moving the high-quality feedback data to an external environment for dedicated analysis.
- **Simulated Retraining (/admin/retrain):**
- Simulates a training pipeline (with a 3-second time.sleep).
- Critically, it updates the global **MODEL_VERSION** to a new version (e.g., v1.x), demonstrating the system's ability to handle and track model updates.

INTEGRATION TESTING: ENSURING PRODUCTION READINESS

- **End-to-End Validation:** Successfully verified the entire workflow, from **index.html** data submission through to API response and correct logging in the CSV files.
- **Edge Case Handling:**
- **Security:** Tested unauthorized access to **/admin/stats** (correctly returns 403 Forbidden).
- **Input:** Tested empty/short text inputs (correctly returns 400 Bad Request).
- **Deployment Ready:**
- **Dependencies:** All dependencies are documented in **requirements.txt**.
- The use of standard libraries and Fast API simplifies containerization (e.g., Dockerisation) for easy cloud deployment.

System Integration Testing

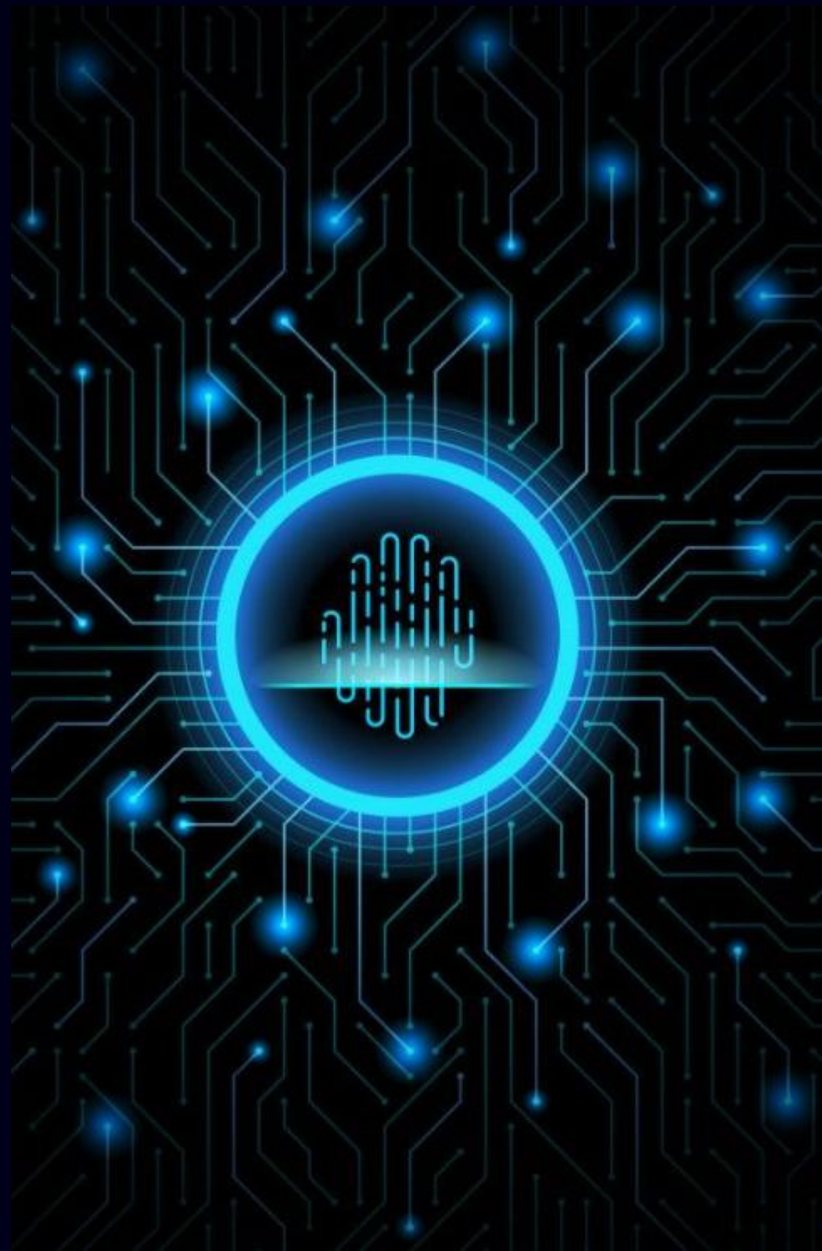
Milestone 3 Conclusion and Future Vision

✅ **Backend:**
Production-grade, secure, and performant API built.
•✅ **Frontend:** Intuitive UI integrated with dynamic result display and feedback.
•✅ **Operational:** Full logging and feedback loop established for continuous data collection.

•**Future Scope (Beyond M3):**
1.Cloud Deployment: Migrate the application to a cloud service (e.g., AWS EC2/Lambda or Azure App Service).
2.Database Migration: Replace CSV files (.logs, .flagged) with a robust database (e.g., PostgreSQL or MongoDB) for better scalability and query performance.
3.Automated Retraining: Implement a CI/CD pipeline that automatically pulls feedback data, retrains the model, and updates the production model file (.pkl) with zero downtime.

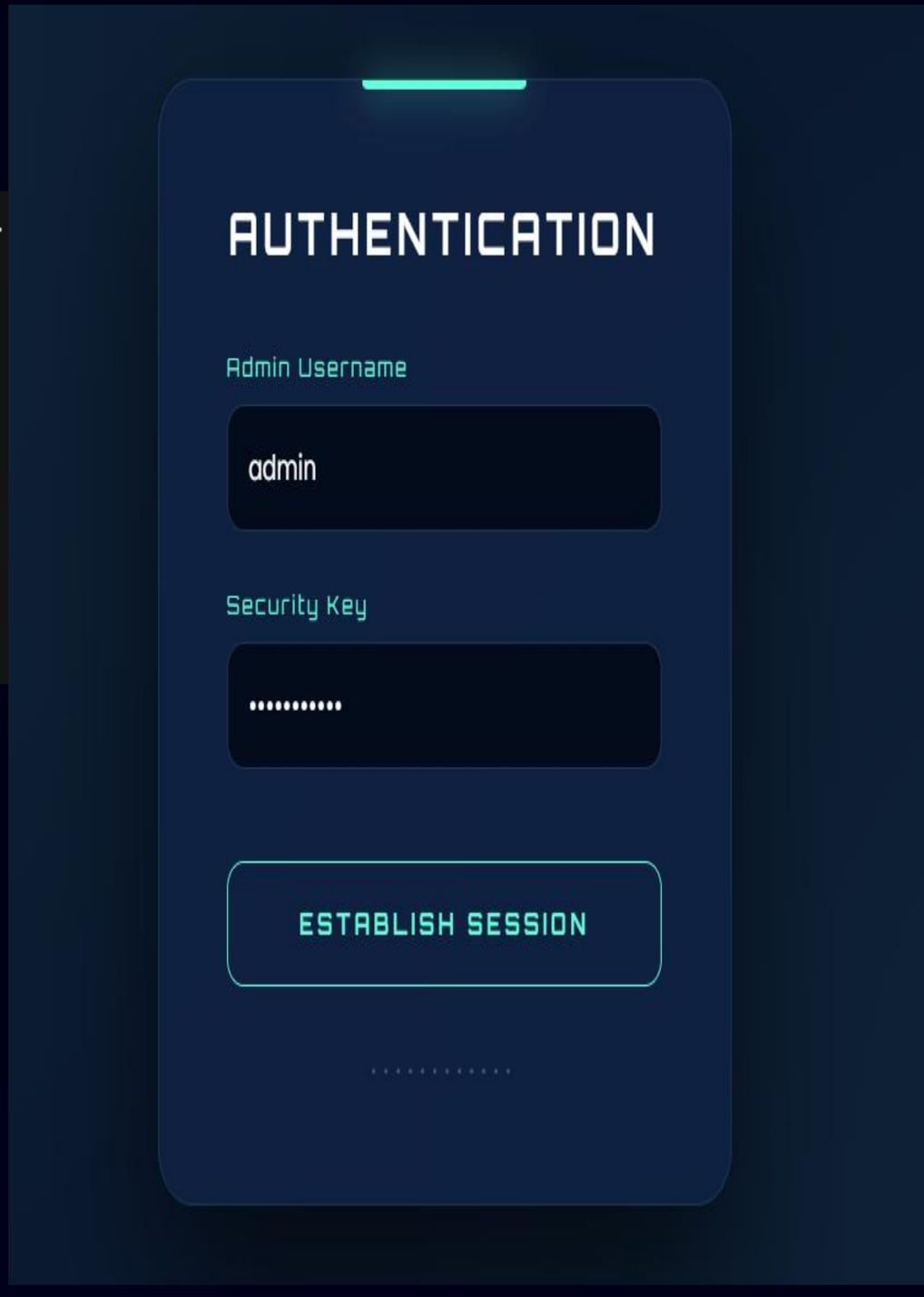
Milestone 4 Strategy – Product Maturity

- **Transition:** Moving from a functional prototype to a secure, enterprise-ready application.
- **Focus Areas:**
  - **Security:** Protecting sensitive admin operations.
  - **Maintenance:** Managing the Model Lifecycle.
  - **Data Integrity:** Implementing Audit Trails.
  - **User Experience:** Closing the feedback loop.



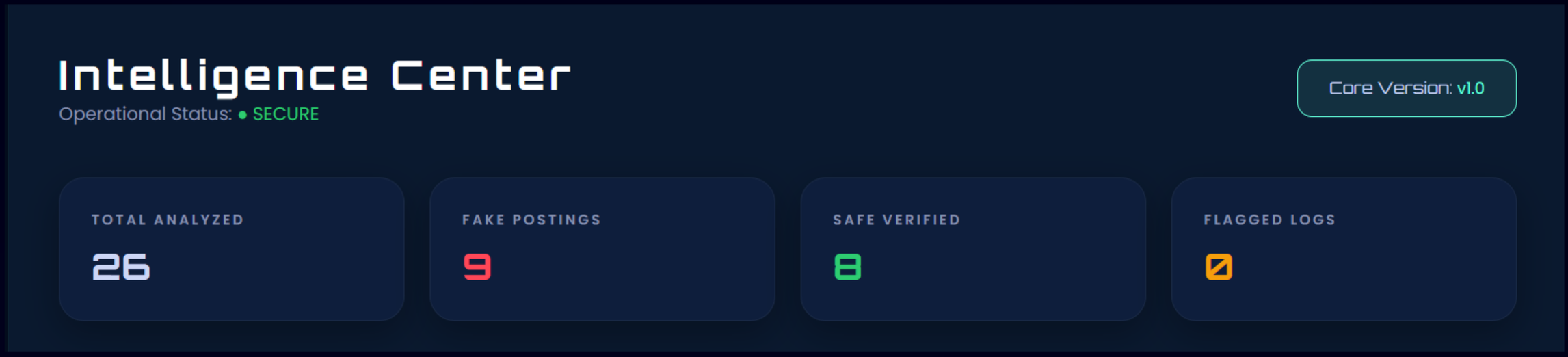
Security Layer – JWT Authentication

- **The Problem:** Administrative endpoints (like retraining) should not be public.
- **The Solution: JSON Web Tokens (JWT).**
- **Workflow:**
  1. Admin logs in with credentials.
  2. Backend issues a cryptographically signed Token.
  3. The browser stores this token to verify all future Admin requests.



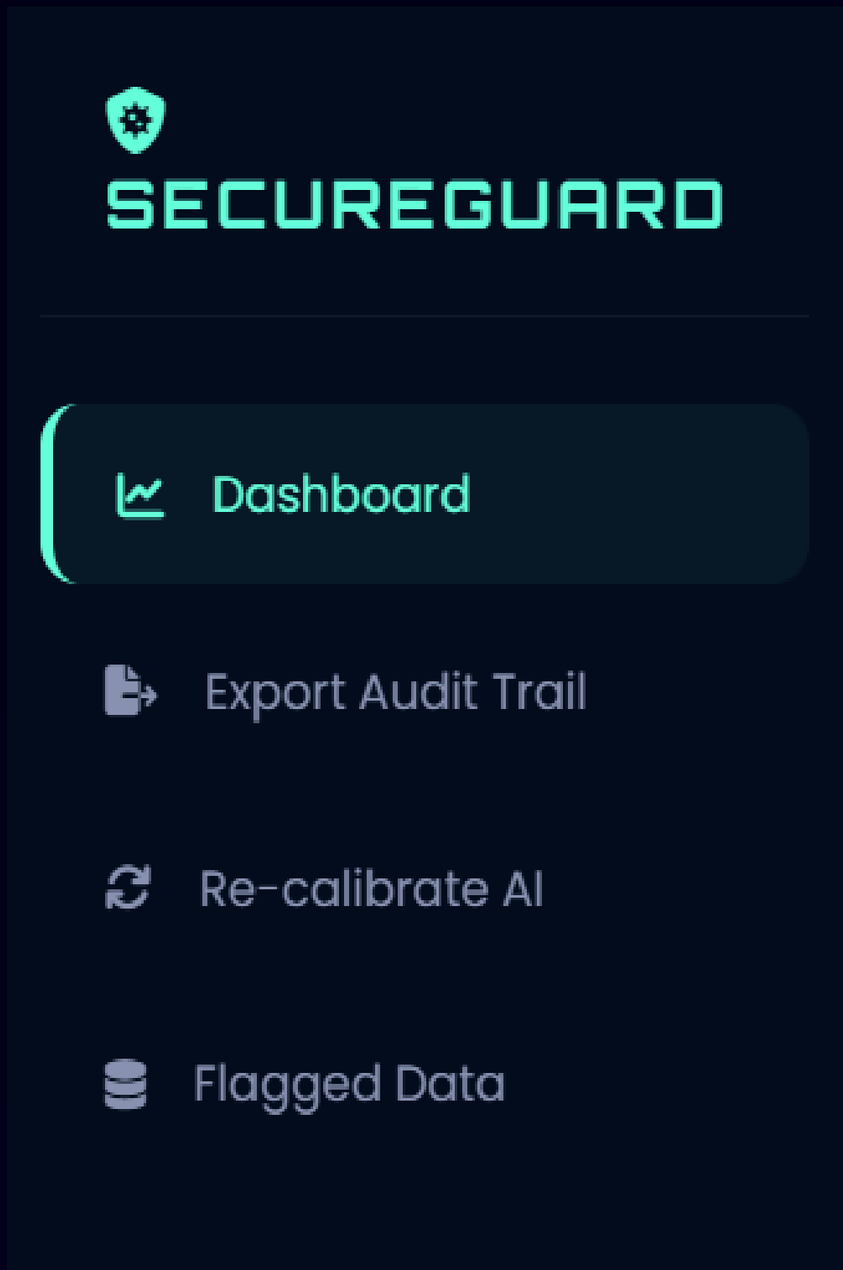
Admin Dashboard – Centralized Control

- **Purpose:** A private UI for system monitoring.
- **Key Metrics Displayed:**
  - Total Jobs Scanned vs. Fraudulent detected.
  - System Health & API Response Times.
  - Dynamic Visualization using charts to track fraud trends.



Audit Trails – System Transparency

- **Implementation:** Permanent logging of all activities in `prediction_logs.csv`.
- **Data Captured:**
  - Timestamp of scan.
  - Snippet of job description.
  - AI Confidence Score (Probability).
- **Value:** Provides a "paper trail" for compliance and performance auditing.



SECUREGUARD

- Dashboard
- Export Audit Trail
- Re-calibrate AI
- Flagged Data

Terminate Session

Intelligence Center

Operational Status: • SECURE

Core Version: v1.0

TOTAL ANALYZED	FAKE POSTINGS	SAFE VERIFIED	FLAGGED LOGS
26	9	8	0

Detection Trends

Fraud Distribution

Human-in-the-Loop – Flagging Mechanism

- **Feature:** Users can "Flag" a result if they believe the AI is wrong.
- **Backend Storage:** Flagged data is saved to `flagged_jobs.csv`.
- **Impact:** This converts real-world mistakes into high-quality training data for future versions.

Model Versioning & Maintenance

- **Challenge:** AI models can become outdated as scammers change tactics.
- **Solution:** Global `MODEL_VERSION` tracking.
- **Action:** Every time the model is updated, the version increments (e.g., v1.0 to v1.1), allowing admins to track improvements over time.

SECUREGUARD

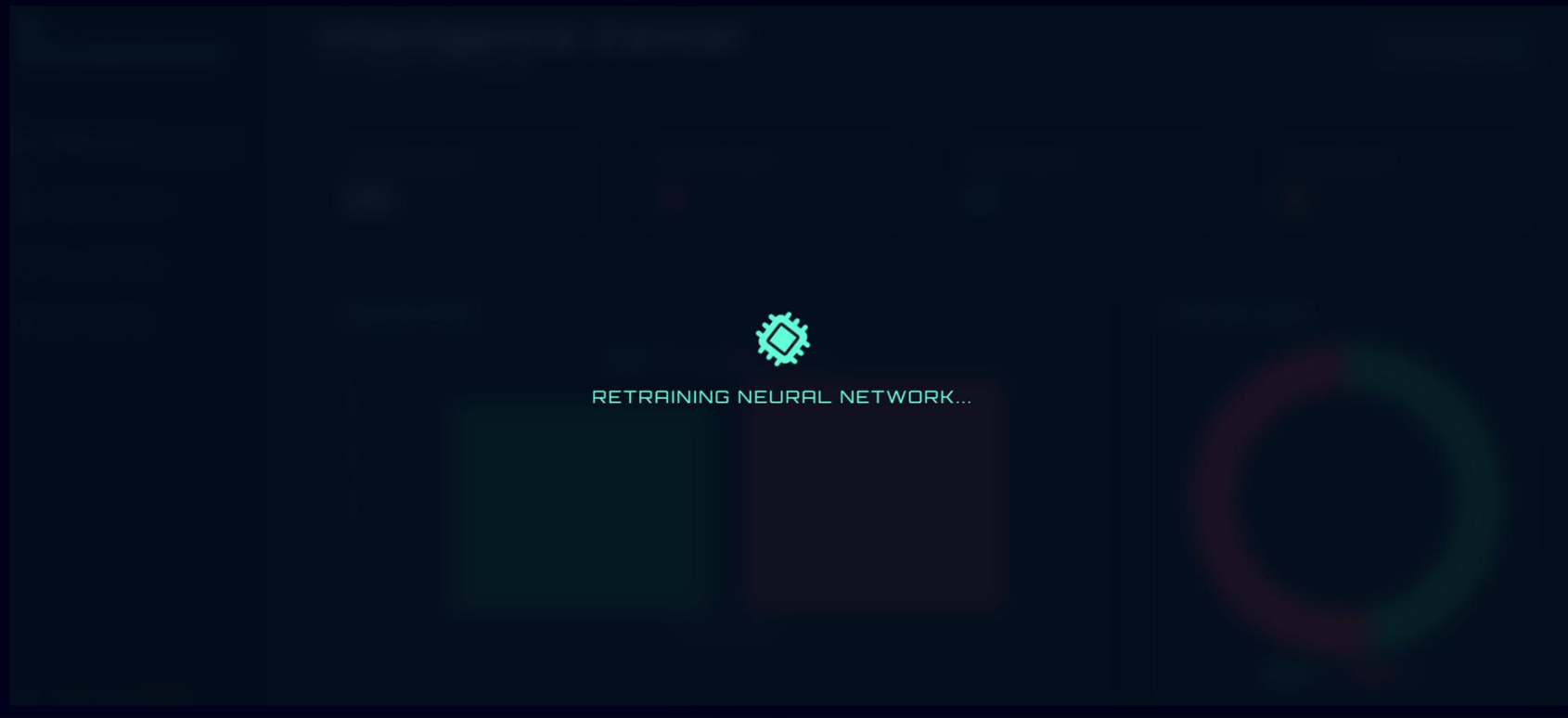
Intelligence Center

Operational Status: • SECURE

Core Version: v1.9

Automated Retraining Pipeline

- **Feature:** One-Click Retraining via the Admin Panel.
- **Technical Process:**
  - Triggers the `/admin/retrain` endpoint.
  - Backend pulls the latest data from `flagged_jobs.csv`.
  - The model "learns" from new patterns and restarts the service.



Data Portability – Administrative Export

- **Feature:** "Export to CSV" functionality.
- **Utility:** Allows administrators to download the entire audit trail and flagged data with one click.
- **Use Case:** External reporting, manual HR review, and offline deep-learning research.

Error Handling & Robustness

- **Input Validation:** Handling empty strings or non-English text gracefully.
- **Security Error Codes:** Implementing `401 Unauthorized` for expired sessions.
- **Graceful Degradation:** The UI remains functional even if the backend is busy retraining.

Future Scope & Conclusion

- **Future Scope:**
  - **Cloud Scaling:** Moving from CSV files to a SQL/NoSQL Database (PostgreSQL/MongoDB).
  - **Real-time Scraping:** Integrating with LinkedIn/Indeed APIs for live scanning.
- **Final Conclusion:** SecureGuard successfully bridges the gap between raw Machine Learning and a secure, manageable software product.

THANKYOU